

Quantus

Double-Bottom Chart-Pattern Scanner & Alert Bot
Design, Backend Architecture & Trading Concepts

Technical Design Document — generated 2026-07-06

Quantus watches a configurable watchlist (the Magnificent 7 by default) on the daily and weekly timeframes, algorithmically detects the **double-bottom** reversal pattern, waits for a volume-backed bullish breakout above the neckline to confirm it, and then pushes an alert — with an annotated candlestick chart — to Telegram and Discord. It includes an interactive Telegram bot for managing the watchlist and a full backtesting engine (with parameter sweeps and out-of-sample validation) for measuring the strategy's historical edge.

1. What Quantus Is

Quantus is a personal, single-user quantitative trading **alert** system. It does not place trades; it surfaces high-quality technical setups and leaves the execution decision to the human. The design goal is a clean, testable, deterministic pipeline that a trader can trust, inspect, and tune against historical data.

At a glance

- **Data:** free end-of-day OHLCV from Yahoo Finance (via the `yfinance` library).
- **Universe:** a watchlist stored in SQLite, seeded from config (AAPL, MSFT, GOOGL, AMZN, NVDA, META, TSLA), editable live from Telegram.
- **Timeframes:** daily and weekly, scanned independently.
- **Pattern:** double bottom, confirmed by a bullish, volume-backed close above the neckline.
- **Delivery:** Telegram + Discord, each alert carrying an annotated chart image.
- **Scheduling:** a once-daily scan run by the hermes cron scheduler; an always-on listener for instant Telegram interaction.
- **Validation:** a walk-forward backtester and a parameter sweep with in-sample / out-of-sample splitting.

The codebase is Python 3.11, organized as the `ddbot` package, with 53 automated tests. Every component is behind an interface (data provider, alerter, pattern) so pieces can be swapped without rewrites.

2. Fundamentals: The Concepts Behind the Bot

This section explains the technical-analysis and quantitative ideas the bot relies on, so the logic in later sections is meaningful.

2.1 The double bottom (and the neckline)

A **double bottom** is a bullish reversal pattern shaped like a 'W'. After a decline, price makes a low (Bottom 1), rebounds to an interim peak, falls again to roughly the same level (Bottom 2), then rallies. The two bottoms mark a support level buyers defended twice. The **neckline** is the interim peak between the bottoms — the resistance that price must break to confirm the reversal. A close above the neckline signals that buyers have overwhelmed the sellers who were capping the rebound.

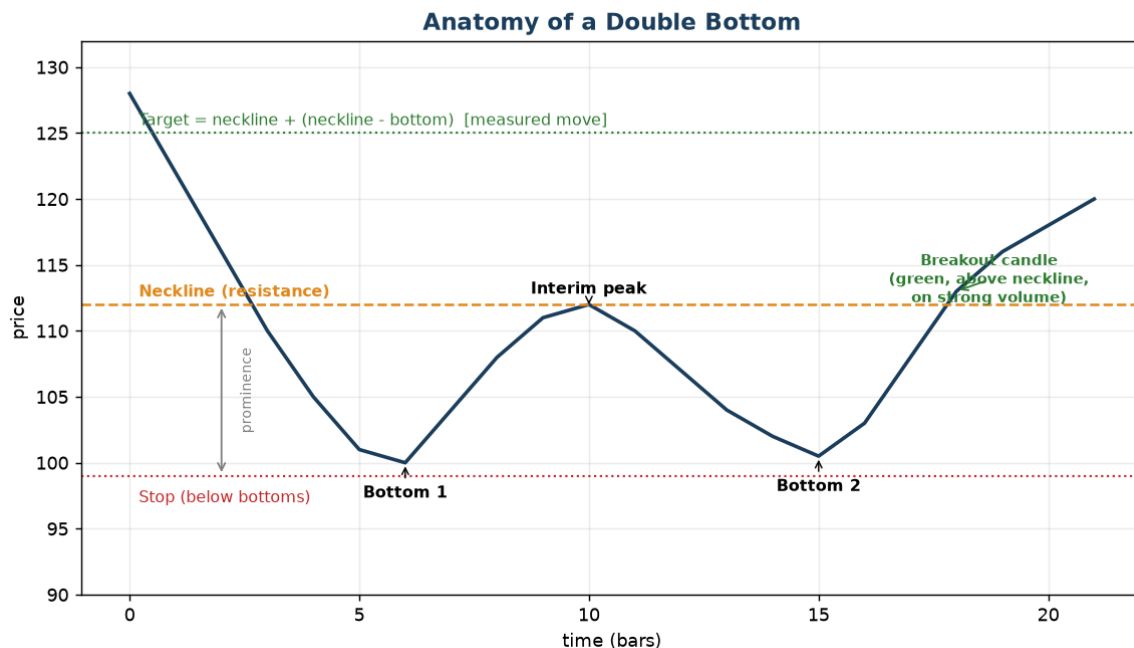


Figure 1. The anatomy of a double bottom: two bottoms at a shared support, the neckline (resistance) between them, prominence (depth), and the confirming breakout. The stop sits below the bottoms; the measured-move target projects the pattern height above the neckline.

2.2 Swing points (fractals)

To find the bottoms and the neckline objectively, the bot detects **swing points**. A bar is a **swing low** if its low is the lowest within a symmetric window of k bars on each side (a 'fractal'); a swing high is the mirror image. This turns a subjective visual pattern into a precise, reproducible rule.

2.3 Prominence

Not every wiggle is a real 'W'. **Prominence** measures how far the neckline sits above the bottoms as a percentage. Requiring a minimum prominence (default 5%) discards shallow, sideways noise and keeps structures with genuine depth — the single most effective false-positive filter.

2.4 Confirmation and no-repaint discipline

A pattern is only a **candidate** until price actually breaks out. Confirmation requires the first candle after Bottom 2 to close above the neckline (plus a small buffer) as a green (bullish) candle. Crucially, the bot acts only on **closed** bars — the still-forming current bar is dropped — because a forming bar can 'repaint' (change) before it closes. This avoids the classic backtest-vs-live discrepancy where a strategy looks good only because it peeked at data that did not yet exist.

2.5 Volume confirmation

A breakout on thin volume often fizzles. The bot requires the breakout candle's volume to be at least a multiple (`volume_factor`) of the recent average volume. Genuine reversals tend to attract participation; the volume gate filters low-conviction breaks.

2.6 Measuring an edge: R-multiples, expectancy, profit factor, drawdown

- **R (risk unit)**: the distance from entry to the protective stop. A trade that gains twice its risk is $+2R$; one that hits the stop is $-1R$. Expressing results in R makes trades comparable across prices and position sizes.

- **Expectancy (average R):** the mean R per trade — the average profit (in risk units) you expect from each signal. Positive expectancy is the definition of an edge.
- **Win rate:** the fraction of trades that are profitable. High win rate with small average R is common for target-based exits and is not automatically good or bad on its own.
- **Profit factor:** gross winning R divided by gross losing R. Above 1.0 is profitable; ~1.3 is modest, ~2+ is strong.
- **Max drawdown:** the largest peak-to-trough drop of the cumulative-R equity curve — a measure of the pain an approach inflicts before recovering.

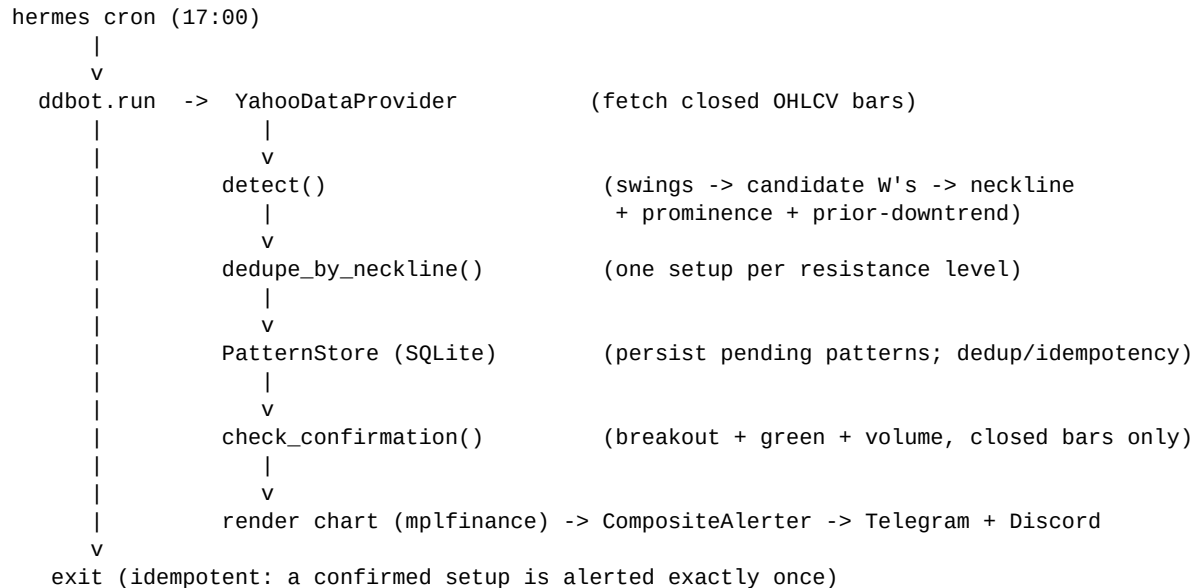
2.7 Walk-forward, in-sample vs out-of-sample, and overfitting

Overfitting (curve-fitting) is tuning parameters until they look perfect on past data, only to fail live. Two defenses are built in. **Walk-forward** testing replays history bar by bar, only ever using data that existed at each moment — no look-ahead. **Out-of-sample (OOS)** validation reserves the most recent slice of history, tunes on the older **in-sample** portion, and checks whether the chosen settings still work on the untouched OOS slice. A setting whose edge vanishes out-of-sample was overfit and is discarded.

3. Backend Architecture

The system is a set of small, single-responsibility modules connected by a linear data pipeline. The same detection code powers the live scanner and the backtester, so what you measure is what you trade.

3.1 Data-flow pipeline (daily scan)



3.2 Module map

Module	Responsibility
data/yahoo.py	Fetch OHLCV from Yahoo; drop the forming bar; symbol validation/normalization.
patterns/swings.py	Swing-high / swing-low (fractal) detection.
patterns/double_bottom.py	detect(), dedupe_by_neckline(), check_confirmation() (incl. volume gate).
patterns/base.py	DoubleBottom data model, PatternState, stable pattern_id, stop reference.
state/store.py	SQLite: patterns, watchlist, key/value (Telegram offset). WAL for shared access.
alerts/*	Telegram, Discord, composite fan-out, message formatter.
charts/chart.py	Annotated candlestick PNG (bottoms, neckline, breakout) via mplfinance.
engine.py	Orchestrates fetch -> detect -> persist -> confirm -> alert per ticker/timeframe.
run.py	CLI entry for the daily scan; loads .env and config.
sync.py / listen.py	Telegram watchlist control: one-shot poll and always-on long-poll listener.
watchlist.py	Deterministic add/remove/list CLI (also used by hermes/qwen).
backtest/*	Walk-forward engine, metrics, parameter sweep, CLI.

3.3 State and idempotency

All state lives in one SQLite file. Detected patterns are stored with a stable `pattern_id = hash(ticker, timeframe, bottom dates)`. Confirmation often arrives on a later run, so patterns persist between runs; an alerted flag guarantees each setup fires exactly once. This makes every scheduled run safe to repeat.

4. The Detection Algorithm, Step by Step

Given a frame of closed OHLCV bars for one ticker and timeframe:

- **1. Swing lows:** mark every bar whose low is the minimum over $\pm$ swing_k bars.
- **2. Candidate pairs:** for each pair of swing lows (B1 before B2), require their spacing to be within [min_bars_between, max_bars_between] and their lows within bottom_to_l_pct of each other.
- **3. Neckline:** take the highest high strictly between the two bottoms; require its prominence above the bottoms to be at least min_prominence_pct.
- **4. Prior downtrend (optional):** require price to have declined into B1 — a reversal needs something to reverse.
- **5. Dedup:** cluster candidates whose necklines sit within $\sim 2\%$ and keep only the strongest (deepest, most symmetric), so one breakout is one setup, not many.
- **6. Persist:** store surviving structures as DETECTED (pending).
- **7. Confirm:** the first later closed candle that closes above neckline $\times (1 + \text{buffer})$, is green, and carries volume $\geq \text{factor} \times \text{average}$ $\rightarrow$ CONFIRMED (alert). A close below the lower bottom, or expiry after max_bars_between, invalidates the pending pattern.

AAPL 1d — Double Bottom confirmed



Figure 2. A real double bottom the bot detected on AAPL (daily). The two blue markers are the bottoms, the dashed orange line is the neckline, and the green line marks the confirming bullish breakout candle. This is the exact chart image attached to the alert.

4.1 Tunable thresholds (config/config.yaml)

Parameter	Default	Meaning
lookback_bars	90	Detection window scanned per ticker.

swing_k	3	Bars each side required to qualify a swing point.
bottom_tol_pct	0.03	Max relative gap between the two bottom lows (3%).
min_prominence_pct	0.05	Neckline must sit $\geq$ 5% above the bottoms.
min_bars_between	5	Bottoms not too close.
max_bars_between	50	Bottoms not too far; also pending-pattern expiry.
neckline_buffer_pct	0.001	Breakout must clear the neckline by 0.1% (noise guard).
require_prior_downtrend	true	Require a decline into Bottom 1.
require_volume_confirmation	true	Enforce the breakout volume gate.
volume_avg_window	20	Bars used for the average-volume baseline.
volume_factor	1.0	Breakout volume $\geq$ this x average (raise $\sim$ 1.5 for stricter).

A note on tooling: chart patterns like the double bottom are **not** provided by TA-Lib or pandas-ta (those cover indicators and single-candle patterns). The structural detection here is custom logic built on NumPy/pandas.

5. Alerting, Charts, Interaction & Scheduling

5.1 Alerts and charts

On confirmation the bot renders an annotated candlestick chart (mplfinance): the two bottoms marked, the neckline drawn, and the green breakout candle highlighted, with a volume panel. A composite alerter fans the message + chart out to Telegram (sendPhoto) and Discord (webhook file upload); a failure on one channel never blocks the other. Each alert includes the neckline, both bottoms, the breakout close, and a suggested stop reference below the bottoms.

5.2 Interactive watchlist (Telegram)

The watchlist lives in SQLite and is editable from Telegram via a fixed two-button keyboard ([+ Add] / [- Remove]) and the commands /list, /add SYMBOL, /remove SYMBOL. New symbols are validated against Yahoo before being added, and only the owner's chat id is honored. Because Telegram cannot wake a stopped program, an **always-on long-polling listener** (kept alive by launchd, auto-restart on crash and boot) provides instant responses. A one-shot polling variant also exists for cron-based operation; only one may poll a bot token at a time.

5.3 Scheduling (hermes) and a macOS gotcha

The daily scan is a one-shot invoked by the hermes cron scheduler at 17:00 local time (Malaysia). Because the bot only acts on closed bars, exact timing is not critical. One practical lesson learned: macOS privacy protection (TCC) blocks launchd agents from executing shell scripts located in ~/Desktop; the fix is to have launchd run the Python interpreter directly (mirroring an existing working service) rather than a wrapper script.

6. Backtesting & Tuning

6.1 Methodology

The backtester replays history through the **exact same** detect() and check_confirmation() code the live bot uses, walking forward bar by bar so there is no look-ahead. The first bar a pattern confirms is the entry. Each trade is then simulated forward:

- **Entry:** the breakout candle's close.
- **Stop:** just below the lower of the two bottoms; risk = entry - stop.
- **Target (default):** the 'measured move' — neckline + (neckline - stop), i.e. the pattern height projected up. (A fixed reward:risk target is also available.)
- **Exit:** whichever of stop or target is hit first (stop assumed first if both hit in one bar, conservative), else a time-exit after max_holdBars.

Results are reported in R and aggregated into win rate, expectancy, total R, profit factor, and max drawdown, per ticker and overall. Caveats: the model ignores slippage, commissions and dividends, assumes one position per signal, and applies no position sizing — results are indicative, not guarantees.

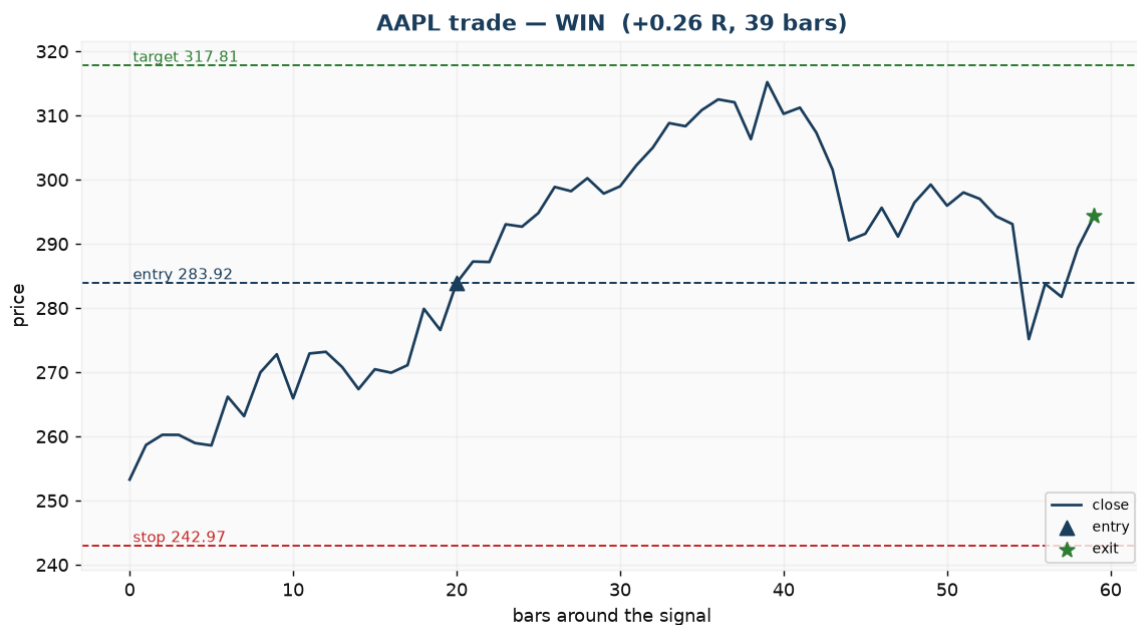


Figure 3. One backtested AAPL trade: entry at the breakout close, stop below the bottoms, and the measured-move target. The star marks the exit; the result is recorded in R (reward relative to the risk taken).

6.2 Measured results (Magnificent 7)

Timeframe	Period	Trades	Win %	Profit factor	Expectancy
Daily	2022-2026 (~4y)	157	68%	1.27	+0.08 R
Weekly	2015-2026 (~11y)	69	70%	1.47	+0.13 R

A real but modest positive edge, with the weekly timeframe cleaner than daily — consistent with the principle that higher timeframes carry less noise.

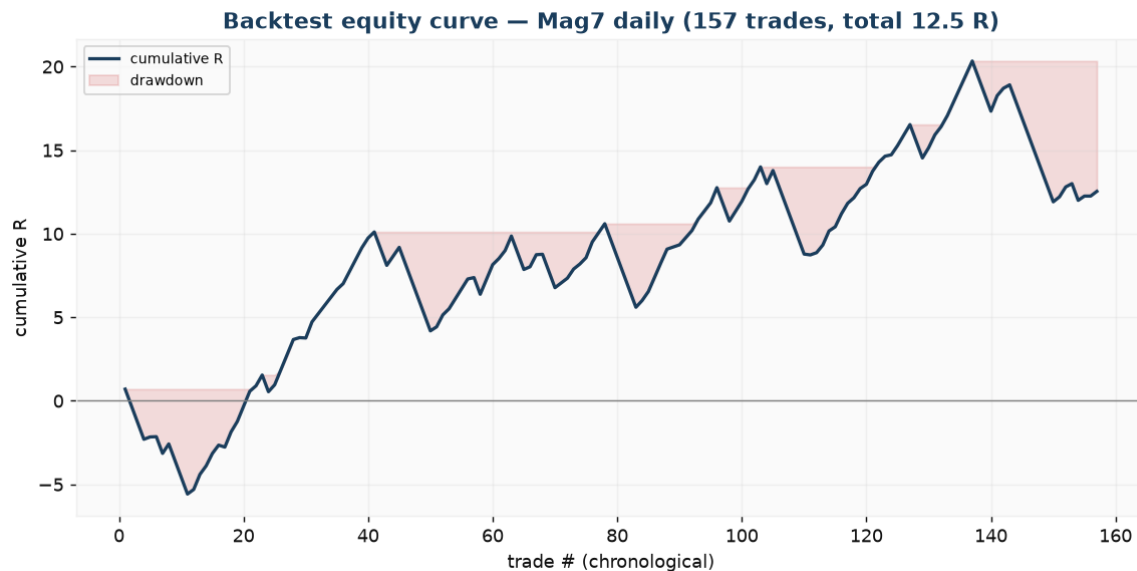


Figure 4. Cumulative-R equity curve across the Mag7 daily backtest (trades in chronological order). The shaded band is drawdown — how far the running total sits below its prior peak.

6.3 Parameter sweep with out-of-sample guard

The sweep grid-searches detection thresholds, backtests each combination, and splits the trades into in-sample (older) and out-of-sample (newest 30%) by entry date. It ranks by an in-sample objective while displaying out-of-sample side by side, so overfit settings are exposed. Example daily result:

volume_factor	min_prominence	IS profit factor	OOS profit factor	Verdict
2.0	0.08	3.86	0.71	Overfit (OOS collapses)
2.0	0.05	3.71	3.01	Holds out-of-sample
1.5	0.05	1.50	2.17	Holds
1.0	0.05 (current)	1.27	1.28	Stable, weakest edge

The lesson the tool teaches: the best in-sample number (2.0/0.08) is a trap, but a broad plateau (volume_factor 1.5-2.0 with min_prominence 0.05) improves the edge and survives out-of-sample. A sweep finds candidates; it does not prove an edge.

Parameter sweep — profit factor (higher = greener). Compare IS vs OOS to spot overfitting.

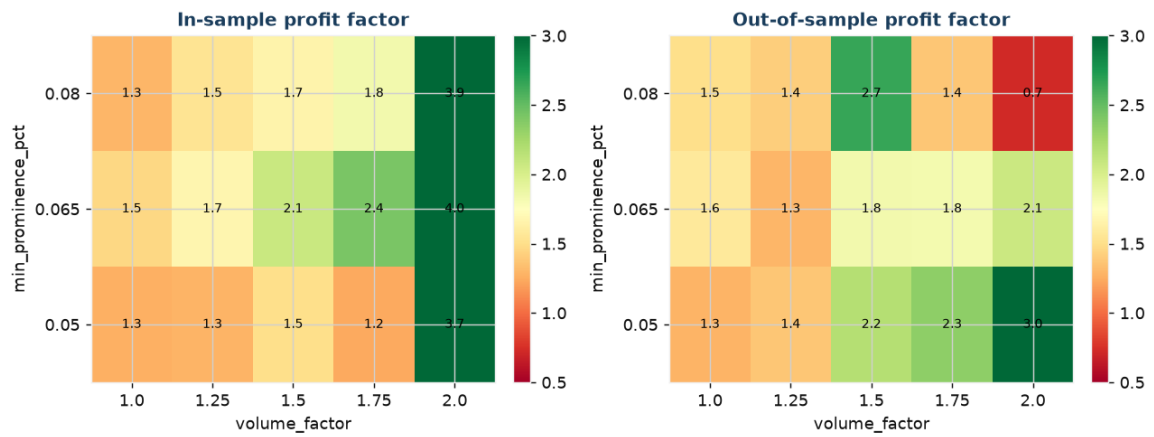


Figure 5. Profit factor across a volume_factor x min_prominence grid, in-sample (left) vs out-of-sample (right); greener is better. The volume_factor=2.0 column looks superb in-sample, but the top cell (2.0 / 0.08) turns red out-of-sample — overfit. The settings that stay green in BOTH panels are the trustworthy ones.

7. Development Timeline

The project was built incrementally, each step verified with tests and live runs before the next.

Phase	What was built
1. MVP	Daily double-bottom detect + confirm; Telegram + Discord alerts; SQLite state; idempotent daily run.
1b. Expansion	Magnificent-7 watchlist; daily + weekly timeframes; annotated chart images in alerts.
Scheduling	hermes cron for the daily scan; .env secret loading; timezone-correct timing.
1.5 Watchlist UX	Watchlist moved into SQLite; Telegram commands + buttons; deterministic CLI for hermes/qwen.
Listener	Always-on long-polling listener (launchd) for instant Telegram interaction.
Quality: dedup	Per-neckline dedup: collapsed ~26 backlog alerts to ~14 (one per setup).
Quality: volume	Volume-confirmation gate on the breakout candle.
Backtesting	Walk-forward engine + metrics + CLI; measured the edge on 4-11 years of history.
Parameter sweep	Grid search with in-sample/out-of-sample validation to tune thresholds safely.
Documentation	This design document.

8. Risks & Limitations

- **False positives are inherent** to pattern detection; the prominence, similarity, dedup and volume filters mitigate but never eliminate them.
- **Data quality:** yfinance is an unofficial, free feed with no SLA; it can be delayed, gap, or change. Prices are split/dividend-adjusted.
- **Modelling gaps:** the backtest ignores slippage, commissions, dividends and position sizing; live results will differ.
- **Overfitting risk** when tuning; always prefer settings that hold out-of-sample.
- **Not financial advice / not automated execution:** Quantus alerts; a human decides. Manual confirmation before trading is intentional, given the probabilistic nature of the signals.

9. Roadmap

- **Apply tuned thresholds** from the sweep (e.g. volume_factor 1.5) for higher-quality alerts.
- **Risk management & position sizing:** convert R-multiples into dollar equity curves and a suggested size per alert (ATR or fixed-fractional).
- **More patterns:** inverse head-and-shoulders, triple bottom — reusing the swing/neckline machinery.
- **Multi-timeframe confirmation:** only alert a daily setup when a weekly setup aligns.
- **Fundamental overlay** and, much later, semi-/fully-automated execution with hard guardrails.

Quantus — technical design document. Generated programmatically from docs/generate_pdf.py; regenerate to keep in sync with the codebase.